



UNIVERSITY COLLEGE ROOSEVELT

LANGE NOORDSTRAAT 1, MIDDELBURG, THE NETHERLANDS

BACHELOR OF LIBERAL ARTS & SCIENCES

SENIOR PROJECT

Building Beats

Microtonal Digital Composing in JavaScript, CSS, and HTML

Author:

Vero Naselli

Student Number:

1825208

Supervisor:

Dr. Tamás Vinkó

Second examiner:

Prof. Frank van der Stappen

July 11, 2026

Contents

1	Introduction	2
2	Results	3
2.1	Initial plan	3
2.2	Code base	4
2.3	Current Result	6
2.4	Limitations	8

Chapter 1

Introduction

This Senior Project aims to develop a browser-based microtonal composition platform, containing pre-compiled tunings and intuitive interfaces. The platform will try to provide an option for beginners to the microtonality field to become familiar with a vast variety of tunings and begin composing simple tunes.

The digital platform will be designed to play and record microtonal music, which uses pitches outside the traditional 12-tone equal temperament scale. The platform features over 3000 tuning systems, allowing musicians to explore alternative scales and harmonies, as well as the opportunity to map pitches directly onto keyboard keys in order to compose within the platform. In addition, it contains recording and exporting functionalities, gain control, and custom tonicization. A custom keyboard will be generated at each input selection from the user, and will be dynamically updated at each change in input.

This project aims to bridge the gap between microtonal music and music enthusiasts who are approaching these systems for the first time. Although most of the music in the Western cultural sphere is based on a 12-semitone division of the octave within the equal temperament, there is an immense variety of musical systems and traditions across the world. Often, access to knowledge, resources, and education impedes larger audiences from interacting with such systems, leading to barriers for newcomers to the field who might otherwise greatly enjoy delving into new musicalities.

As such, this platform provides an easy gateway into new sounds and new compositions, offering a streamlined interface, a history and theory section, and a well-defined list of tunings. These features aim to ease the transition into a new landscape, allowing beginners to take the first steps into digital music and microtonality.

Chapter 2

Results

2.1 Initial plan

Following the initial research and material gathering, the plan for the platform included the following features:

- Selection of tunings and temperaments to choose from, with customizable options
- Recording capability
- Import and Export of files
- Visualization of the composition
- Adaptable to any screen device
- Intuitive UI/UX
- History and Tutorial pages

Each feature was added after considering the information gathered during the literature review and my own resources, both in terms of knowledge and time frame. While the scope of the project had to be limited, due to my lack of knowledge in any of the three languages I planned to employ, all key features necessary for basic and complete functionality were present.

Through the Tune.js library, I was able to include an extensive list of ready-made tunings, taken from the Scala archive via Microtuner files compiled by Victor Cerullo. As the Scala program has become a cornerstone for all microtonal digital composers (*Scala Home Page* 2025, *Scala* 2025), it was important to me that my own program could compete with the variety present in other similar projects. Although the .scl format is not supported at the current time, it would be a valuable addition that could further increase the utility of this website.

Recording the musical composition was necessary if the work users executed on my website was to be shared. Amongst the various file formats that were available, WAV or MIDI format were the two best candidates, due to the high fidelity and precision they offered in preserving the original qualities and characteristics of audio data. The main discriminant in choosing either of them was

the compatibility with the logic implemented for audio synthesis in the Synthesizer at the core of the program.

Once recorded, the audio should be downloaded and exported. The matter of importing and exporting files is a fundamental functionality that allows for the sharing and distribution of tuning systems and musical compositions. The goal of this website was to include one avenue of import for tunings or musical files to be implemented in a composition, and one avenue of export to export musical files to other platforms.

The matter of importing a tuning is somewhat alleviated by the variety of tunings already present, and as such was not considered as urgent as other features listed here. Currently, it remains a useful addition that will be implemented in further work. On the other hand, exporting musical files was prioritized as one of the key features of this website and was to be implemented through either of two avenues, either by including MIDI support or WAV support.

Other features that I considered important to include were audio visualization, adaptive and intuitive UI/UX (user interface and experience), and a history and theory section. These additions would ensure that the composing process is as streamlined and straightforward as possible for users of all skill levels. In order to accomplish that, an audio visualization would also improve the experience of a user looking to understand musical composition for the first time, adding another sensorial layer to the understanding of music and audio. To build on that, the History and Theory and Tutorial sections will be explaining and briefing a new user on the core concepts of microtonality and the use of the main features of the website.

2.2 Code base

The complete code base was written in VS Code, due to the possibility of combining multiple different languages in different files within the same project.

Currently, the full code for this project is available at:

<https://vero-nas.github.io/building-beats/>.

The website is hosted on GitHub Pages, so that the front-end can be public while the back-end is still available to be edited and updated. This solution also aims to solve the issue of browser compatibility and system compatibility for the JavaScript libraries used. In addition to that, the libraries used within the project were originally GitHub projects hosted by individuals who wished to share them. The licenses with which they were published determine that all projects containing the use of these libraries, especially if as extensive as in the case of this Senior Project, be made public to the public. As an additional incentive, GitHub Pages is a free service.

The complete PROJECT folder includes 19 files and 4 folders, with the directory structure outlined in 2.1.

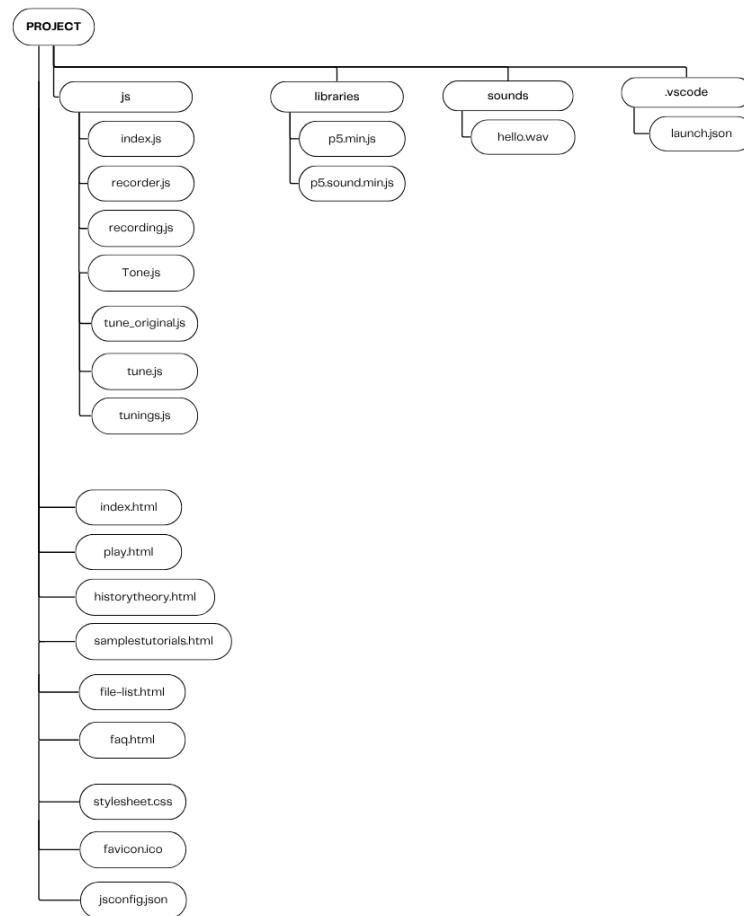


Figure 2.1: Directory structure of codebase for Microtonal Browser Composing

The directory contains four folders. The first folder, "js", contains the libraries `Tone.js`, `tune_original.js`, `tunings.js`, and `recorder.js`, as well as the JavaScript scripts `index.js`, `tune.js`, and `recording.js`. All of these scripts are necessary to run the main functions of the website. The folder "libraries" contains the `p5.js` scripts, which will be necessary to run the audio visualization in the future. The last configuration folder, ".vscode", contains the file that launches the local server.

An additional folder "sounds" contains the file "hello.wav", which will be implemented in the home page (`index.html`) visualization.

Outside of the folders, the directory contains six HTML files and two additional styling files. Each HTML file builds one page of the website: "index.html", "play.html", "historytheory.html", "samplestutorials.html", "faq.html", "file-list.html". Each of these pages is styled by the CSS file "stylesheet.css" and contains the web icon "favicon.ico".

2.3 Current Result

Based on the initial plans, the development process aimed to address each point in order of importance, starting with the audio production, recording, exporting, and ending with the visualization functions, which were all to be achieved using JavaScript. The HTML and CSS programming mainly occupied the beginning of the process and was only updated to suit the needs of the other JavaScript features as they were added. The development process was logged and recorded in two documents: one working log, updated at each work session, and one research collection, containing all necessary resources (Naselli 2025b, Naselli 2025a).

The architecture underlying the project begins with the audio graph. The audio graph describes the audio nodes and the paths taken by the audio data, from source to destination. As shown in 2.2, the audio graph begins with an audio source, proceeds through a Gain node, then splits in parallel to pass through a Recorder node or to the Destination node, which is the audio output method for each device. Once the audio blob has been recorded, the Player feature routes the audio back to the output Destination.

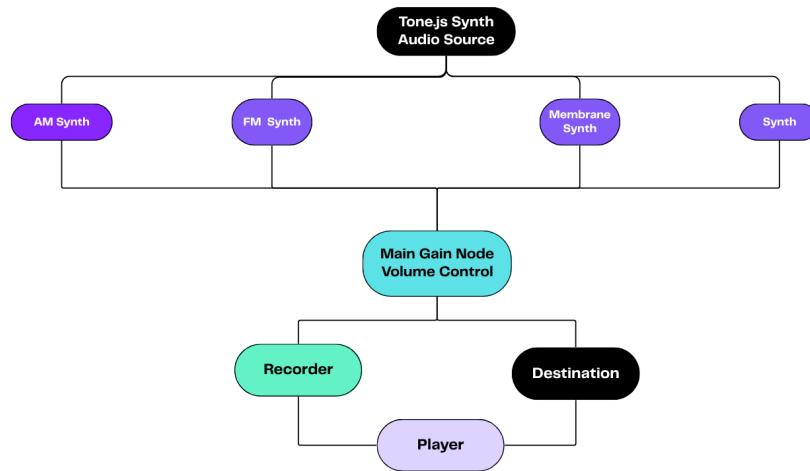


Figure 2.2: Audio routing graph of Microtonal Digital Composing

The only audio manipulation present defined within my project is the Gain node, which controls the decibels (dB) of the signal, which is the level of the audio input (*Gain* — *Tone.js* 2025). It works in tandem with the volume, which controls the level of the output and is controlled by the user for each output device, such as speakers or headphones. Further audio manipulation occurs directly within the Tone.js synthesizers themselves. Each synthesizer contains different routing and produces a different sound texture.

A synthesizer is commonly defined as a machine that generates wave forms and then subjects them to changes to modify the output sound (Vail and Vail 2014).

The oscillator is what originally produces sound within a synthesizer. The oscillator will produce a sound wave with a certain shape, and the different shapes of the sound wave will determine a different sound texture. For example, sine waves are said to have a pure, smooth sound, while cosine waves tend to be a little more complex.

The input produced by the oscillator is then usually fed into an amplitude envelope, which will control its acoustic parameters. One commonly used envelope is the ADSR envelope: it controls the attack, delay, sustain, and release of the signal. Essentially, it decides the onset, decay, and offset of a sound signal in terms of decibels (Vail and Vail 2014).

The most straightforward synthesizer used in this project is the simple Synth, which contains a Tone.js OmniOscillator and an ADSR Amplitude Envelope (*OmniOscillator* — *Tone.js* 2025). The OmniOscillator is able to switch between all available Tone.js Oscillators, and accepts as argument the default sine wave oscillator. This is the initialized waveform for all of the synthesizers.

The AMSynth and the FMSynth use the output of a first Tone.js Synth to modulate the amplitude of a second one. They produce a more complex, dynamic sound, compared to the simple Tone.Synth (*AMSynth* — *Tone.js* 2025, *FMSynth* — *Tone.js* 2025).

The last option available is the MembraneSynth, which produces kick and tom sounds by routing an OmniOscillator through an ADSR envelope (*MembraneSynth* — *Tone.js* 2025).

While the functioning of all of them is very similar, each synthesizer contributes a very different feeling to each scale and tuning.

The other two key components needed to initialize the synthesizer are the tuning type and the tonic frequency. For the purpose of this project, each specific tuning was compiled into an array which contained its name, its description, and the collection of frequencies for one octave of the scale, defined starting from the tonic frequency 261.63 Hz. This is the frequency of the middle C, or C4 pitch, used as a frame of reference for all of the scales. It is important that all the scales are standardized within one reference framework, so that it is possible to transpose them and tonicize them with specific functions defined in Tune.js. Essentially, the standard frequencies of each tuning act as rules upon which the scale is built, and the tonic frequency selected by the user acts as the root of the new scale: using the patterns of intervals and ratios between consecutive and related frequencies within one tuning octave, it is possible to build customized scales based on any frequency given as input. It is also possible to transpose the scale through the octaves, given that the ratios between frequencies are maintained as the standard dictates. All of the tunings are compiled into one dictionary and they are contained in the "tunings.js" file, or in the "tune.js" file as a variable.

As the user selects a scale, a root frequency, and a synthesizer, they are able to customize any tuning of their choice to suit their compositional needs. Once the input has been submitted, a keyboard will be produced. Each step is numbered in ascending order, starting from 1. The user can play the keyboard via both the mouse and the keyboard of the device they are using. At the current time, the key mapping logic is incomplete and will be updated as first priority. More on this is mentioned within the Limitations section.

The user is then able to record the gain and volume of the audio signal, as well as record audio clips of their compositions.

The Recorder functions very simply by recording audio clips from the Web Media Stream and converting them into WAV format from WEBM format.

At its current state, the program achieves most of the objectives established within the initial planning stage: it is able to select a tuning and produce a keyboard with customizable options, it is able to record and export files, it is adaptable to almost any screen and device, and features a simple, intuitive interface, along with educational materials for users. The project achieves its goal of providing a functional, simple portal to learn about microtonal tunings for users of all skill levels and knowledge levels (2.3).

The limitations of this program are discussed further within the Limitations section.

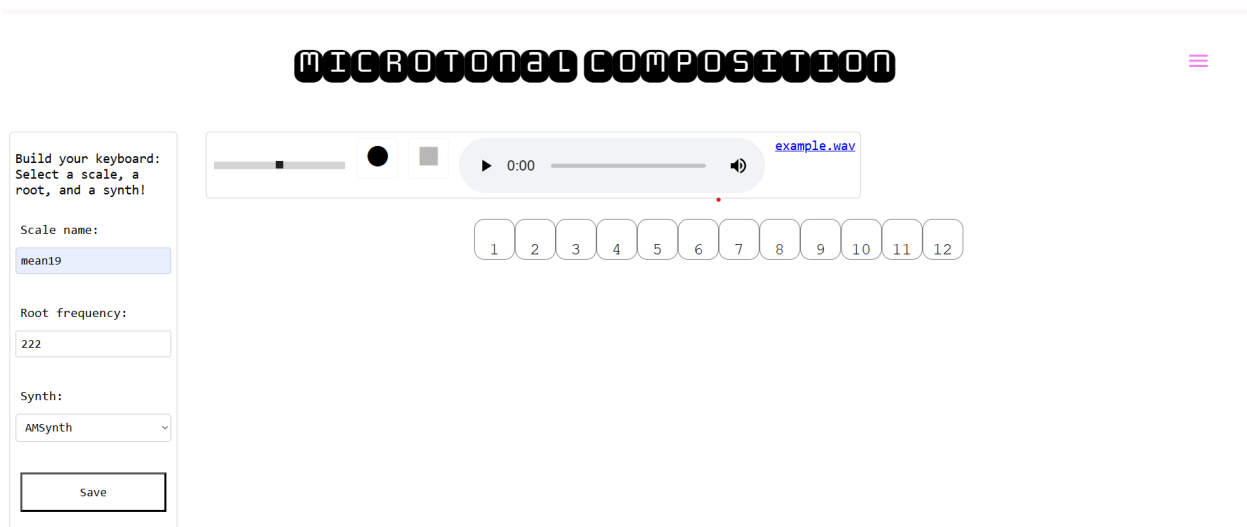


Figure 2.3: Snapshot of an example usage of the Play section of the website.

2.4 Limitations

Given the time frame and my lack of previous knowledge on HTML, CSS, or JavaScript, the project contains limited functionalities. I was able to build it using tutorials and existing implementations; however, there is still much to be added once I have developed my programming skills and I am able to independently define my own frameworks.

The main limitation is incomplete key mappings, due to the restricted number of keys available on a physical keyboard. Scales with more than 50 keys lack key mappings for pitches 51 and above, due to issues in the logic used to record keyboard events in JavaScript, which will be addressed in the coming updates. Ultimately, each scale will have complete and functional key mappings.

Another limitation regarding the keyboard lies in the lack of session storage: once a user changes pages in the website, the input previously submitted and the keyboard are wiped from the session

storage and deleted. Although preliminary logic is in place to address it, further improvements are needed to achieve full effectiveness.

Although an extensive list of tunings is already implemented within the platform, there is no option to import the user's own tuning or to create a custom tuning. This feature is present in other platforms mentioned in the literature review and would improve the compositional flexibility of the program.

Finally, audio visualizations were not present in the final result, although they were included in the original plan. Due to some incompatibility issues between the Gibber.js and P5.js libraries and my own program, the graphics that were planned are currently a work in progress (Roberts 2025, *p5.js* 2025).

Bibliography

- AMSynth* — *Tone.js* (2025). URL: <https://tonejs.github.io/docs/15.1.22/classes/AMSynth.html> (visited on 05/31/2025).
- FMSynth* — *Tone.js* (2025). URL: <https://tonejs.github.io/docs/15.1.22/classes/FMSynth.html> (visited on 05/31/2025).
- Gain* — *Tone.js* (2025). URL: <https://tonejs.github.io/docs/15.1.22/classes/Gain.html> (visited on 05/31/2025).
- MembraneSynth* — *Tone.js* (2025). URL: <https://tonejs.github.io/docs/15.1.22/classes/MembraneSynth.html> (visited on 05/31/2025).
- Naselli, Vero (2025a). *SENIOR PROJECT - Research dump - Documenti Google*. URL: https://docs.google.com/document/d/1aRIWkaMhWJDxj1MR49WL01uurDbXi3fQV_ymIpYyXac/edit?tab=t.0 (visited on 05/30/2025).
- (2025b). *SEPR: To-Do working log - Documenti Google*. URL: https://docs.google.com/document/d/1_5We5mIK2gMEf4dRanyVzWG6k1EkmAneGTjDZZdTokg/edit?tab=t.0 (visited on 05/28/2025).
- OmniOscillator* — *Tone.js* (2025). URL: <https://tonejs.github.io/docs/15.1.22/classes/OmniOscillator.html> (visited on 05/31/2025).
- p5.js* (2025). en. URL: <https://p5js.org/> (visited on 05/30/2025).
- Roberts, Charlie (2025). *Gibber*. URL: <https://gibber.cc/alpha/playground/> (visited on 05/30/2025).
- Scala* (2025). en. URL: <https://tuning.ableton.com> (visited on 05/30/2025).
- Scala Home Page* (2025). URL: <https://www.huygens-fokker.org/scala/> (visited on 05/30/2025).
- Vail, Mark and Mark Vail (Mar. 2014). *The Synthesizer: A Comprehensive Guide to Understanding, Programming, Playing, and Recording the Ultimate Electronic Music Instrument*. Oxford, New York: Oxford University Press. ISBN: 978-0-19-539489-4.